



School of Computer Science and Engineering

Course Title: Software Engineering Lab

Course Code: BCSE301P

Lab Assessment III

Slot:L3-L4

Class Id: VL2025260504501

Due date:16 Feb 2026

Name:Aryan Chopra

Registration No.: 23BCI0144

Ex. No: 6 - Object-Oriented Modelling (Use Case & Class Diagram)

1. **Aim**
2. **Problem Statement**
3. **Use Case Modelling**
 - 3.1 Identification of Actors
 - 3.2 Identification of Use Cases
 - 3.3 Use Case Diagram
4. **Class Modelling**
 - 4.1 Identification of Conceptual Classes
 - 4.3 Class Diagram
5. **Conclusion**

Ex. No: 7 - Object-Oriented Interaction Modelling

1. **Aim**
2. **Problem Statement**
3. **Sequence Diagram**
 - 3.1 Scenario Description (e.g., Booking Appointment)
 - 3.2 Diagram

4. Collaboration Diagram

- 4.1 Interaction Flow Description
- 4.2 Diagram

5. Conclusion

Ex. No: 8 - Package and Deployment Models

1. Aim

2. Problem Statement

3. Package Diagram

- 3.1 Identification of Packages (Layers)
- 3.2 Diagram

4. Deployment Diagram

- 4.1 Identification of Hardware Nodes & Artifacts
- 4.2 Diagram

5. Conclusion

Ex. No: 6 - Object-Oriented Modelling (Use Case & Class Diagram)

Aim: To identify the actors, use cases, and conceptual classes for the "MediConnect" Telemedicine Platform and design the Use Case and Class Diagrams.

Problem Statement: "MediConnect" Telemedicine Platform The system requires object-oriented modelling to represent user interactions and system structure. Patients need to search for doctors, book appointments, and view prescriptions. Doctors need to manage schedules and conduct video consultations. Admins verify credentials. The system must handle secure payments and real-time data exchange.

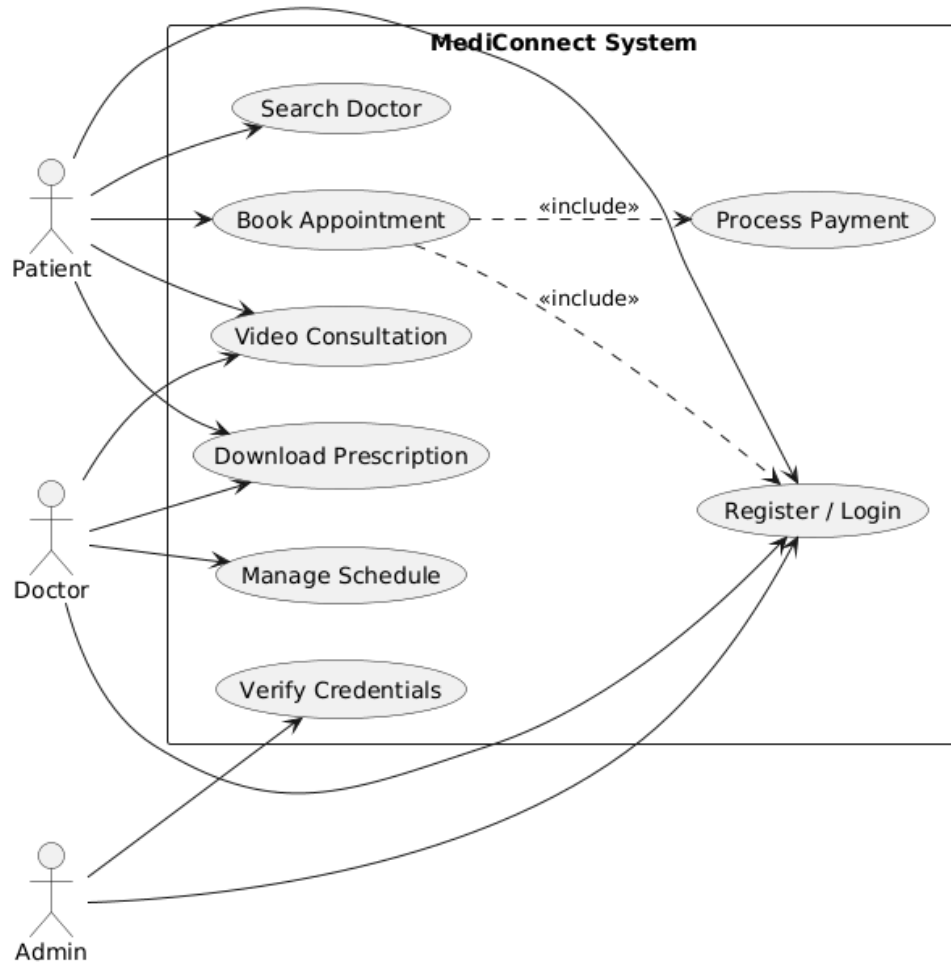
Use Case Model

1. Identification of Actors:

- **Patient:** The primary user who registers, searches for doctors, books appointments, processes payments, and downloads prescriptions.
- **Doctor:** The medical professional who manages availability, conducts video consultations, and issues prescriptions.
- **Admin:** The system overseer who verifies doctor credentials and manages user accounts.

Identification of Use Cases:

- **UC-01 Login/Register:** Authentication for all users.
- **UC-02 Search Doctor:** Patients filter specialists.
- **UC-03 Book Appointment:** Patients reserve a time slot.
- **UC-04 Process Payment:** Handles financial transactions (included in booking).
- **UC-05 Video Consultation:** Real-time session between Patient and Doctor.
- **UC-06 Manage Schedule:** Doctors set their availability.
- **UC-07 Verify Credentials:** Admin approves doctor licenses.

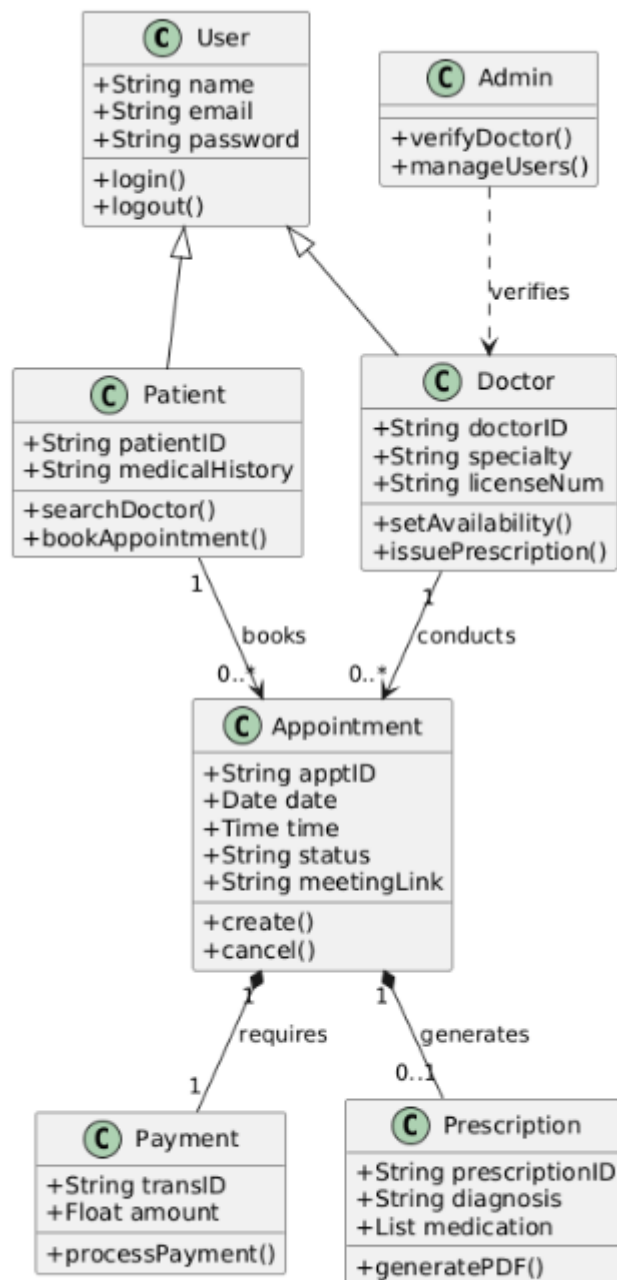


b) Class Diagram

Identification of Conceptual Classes:

- **User (Base Class):** Stores common attributes like Name, Email, Password.
- **Patient (Subclass):** Inherits from User; adds MedicalHistory.
- **Doctor (Subclass):** Inherits from User; adds Specialty, LicenseNumber.
- **Appointment:** Links Patient and Doctor; tracks Date, Time, Status.
- **Payment:** Associated with Appointment; tracks TransactionID, Amount.
- **Prescription:** Associated with Appointment; tracks Diagnosis, Medication.

Class Diagram:



Conclusion

The object-oriented analysis for the "MediConnect" system was successfully completed. By identifying the key actors (Patient, Doctor, Admin) and their interactions, a Use Case model was developed to visualize the system's functional requirements. Furthermore, the conceptual class diagram was designed, defining the essential entities, their attributes, and relationships such as inheritance and composition. This modeling phase establishes a robust structural

foundation for the subsequent development and implementation of the telemedicine platform.

Ex. No: 7 - Object-Oriented Interaction Modelling

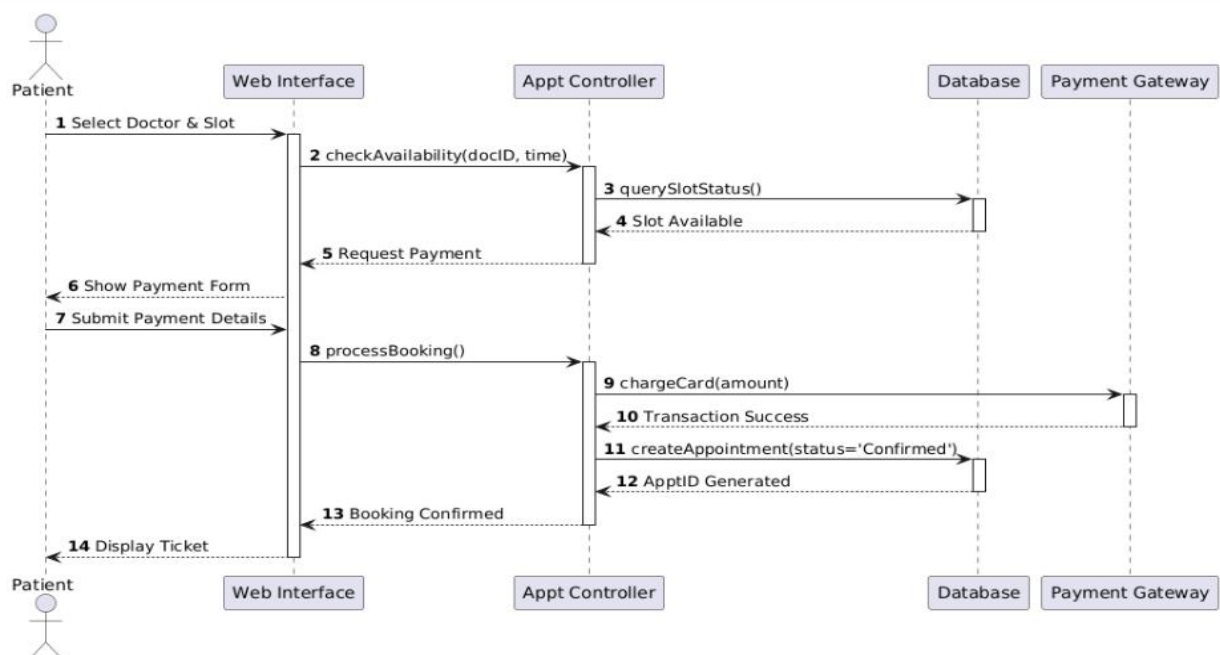
Aim: To model the dynamic behavior of the system using Sequence and Collaboration diagrams for the "**Book Appointment**" scenario.

a) Sequence Diagram

Scenario Description:

1. **Patient** selects a doctor and time slot via the **Web Interface**.
2. **Controller** checks availability in the **Database**.
3. If available, **Controller** requests payment via **Payment Gateway**.
4. Upon success, **Controller** creates the appointment and returns a confirmation ticket.

Sequence Diagram:

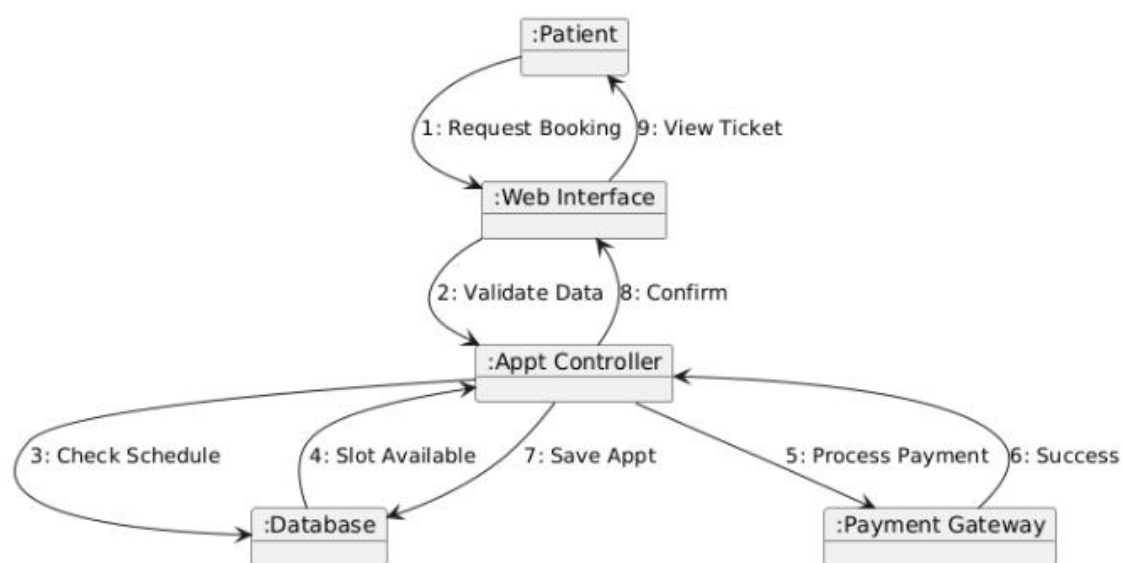


b) Collaboration Diagram

Interaction Flow:

- The **Patient** sends a request to the **Web Interface**.
- The **Web Interface** forwards data to the **Appointment Controller**.
- The **Controller** queries the **Database** for slot status.
- The **Controller** triggers the **Payment Gateway**.
- The **Payment Gateway** returns status to the **Controller**, which then updates the **Database** and notifies the **Patient**.

Collaboration Diagram:



Conclusion

The dynamic behavior of the "MediConnect" system was effectively modeled using interaction diagrams. The Sequence Diagram highlighted the time-ordered message flow for critical scenarios like "Appointment Booking," ensuring logical consistency between the User Interface, Controller, and Database. Additionally, the Collaboration Diagram illustrated the structural organization and message exchange between objects, clarifying the responsibilities of the Appointment Controller as a central coordinator. These models confirm that the system logic is sound and ready for detailed design.

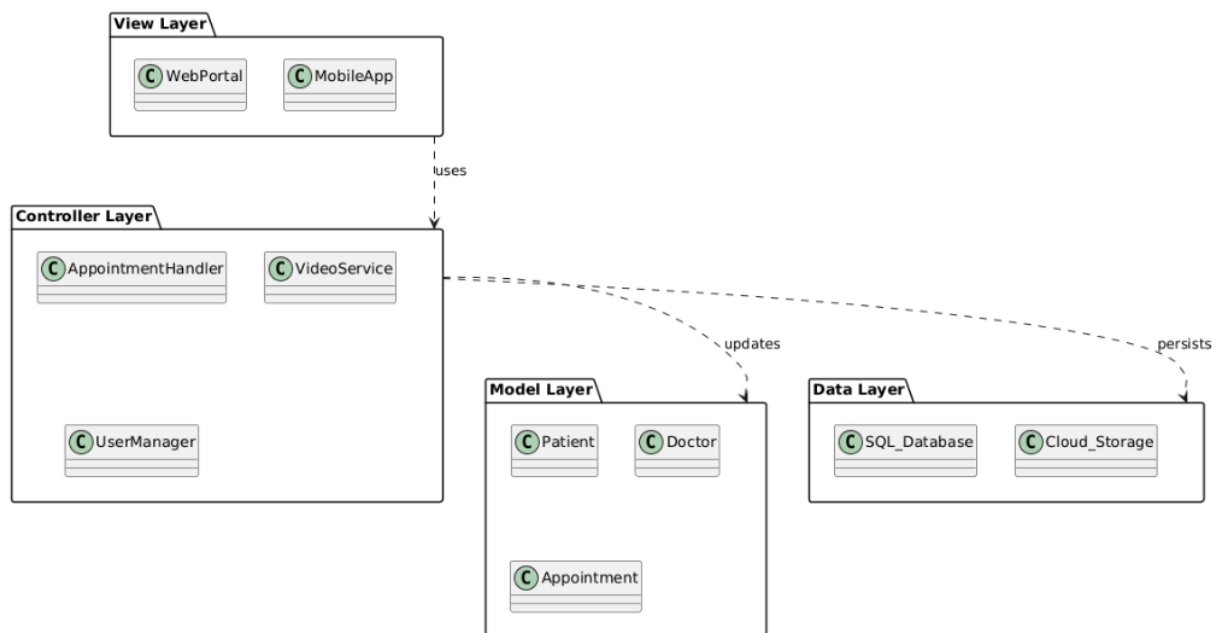
Ex. No: 8 - Package and Deployment Models

Aim: To structure the system into logical packages and map software components to hardware nodes.

a) Package Structure:

- **View Layer:** Contains UI components (WebPortal, MobileApp).
- **Controller Layer:** Contains business logic (AppointmentHandler, UserManager).
- **Model Layer:** Contains data entities (Patient, Doctor, Appointment).
- **Data Layer:** Contains database connections (SQL_Database).

Package Diagram:

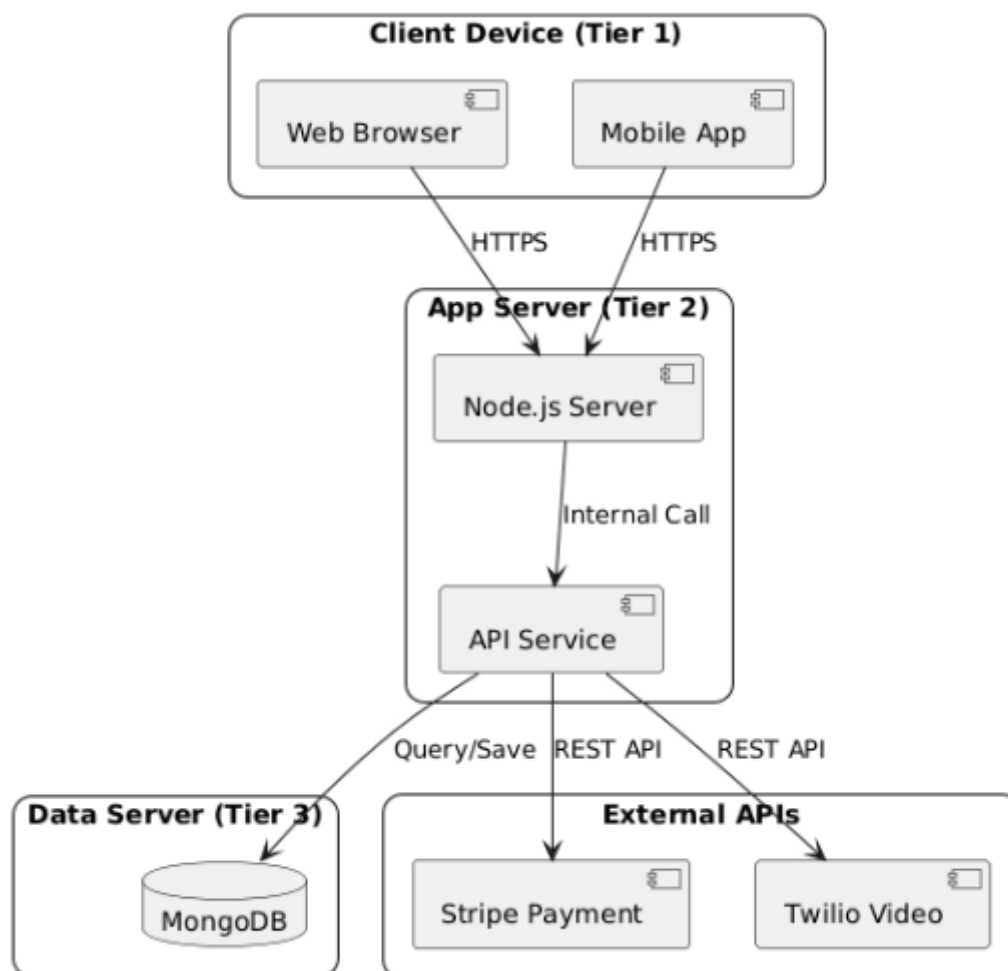


b) Deployment Diagram

Hardware Nodes & Artifacts:

- **Client Device:** Runs the **Web Browser** or **Mobile App**.
- **Application Server:** Runs the **Node.js Server** and **API Service**.
- **Database Server:** Hosts the **MongoDB Cluster**.
- **External Cloud:** Hosts **Stripe API** and **Twilio Video**.

Deployment Diagram:



Conclusion

The architectural design of the "MediConnect" system was finalized through Package and Deployment modeling. The Package Diagram organized the system into logical layers (View, Controller, Model, and Data), ensuring a modular structure that promotes maintainability and separation of concerns. The Deployment Diagram mapped these software components to the physical hardware topology, including client devices, application servers, and external

cloud services. This confirms the system's scalability and readiness for deployment in a cloud-based environment.